

LINUX

TROUBLESHOOTING

FOR BEGINNERS

H A N D B O O K



**FIND
THE PROBLEM**



**UNDERSTAND
THE CAUSE**



**APPLY
THE FIX**



**VERIFY
THE RESULT**

A step-by-step guide to
diagnose and solve real Linux
problems with confidence.



*Think
Diagnose
Solve
Grow*

REAL SCENARIOS

PRACTICAL EXAMPLES

TROUBLESHOOTING STEPS

COMMON MISTAKES

PRODUCTION TIPS



LOGS



SERVICES



STORAGE



NETWORK



PERMISSIONS



PROCESSES

BUILD YOUR LINUX TROUBLESHOOTING SKILLS
ONE PROBLEM AT A TIME

**JUNIOR ENGINEER
TO PRODUCTION**

HOW LINUX TROUBLESHOOTING ACTUALLY WORKS

UNDERSTAND • INVESTIGATE • SOLVE • BUILD CONFIDENCE

1. WHAT IS LINUX TROUBLESHOOTING?



Linux troubleshooting is the process of finding the cause of a problem, understanding why it happened, and applying the right fix so the system works correctly again.

3. WHY RANDOM COMMANDS ARE A BAD HABIT



- Can make the problem worse
- Wastes time
- May hide the real issue
- Can cause unintended changes
- Harder to learn and remember
- Not a professional approach

Troubleshooting is not about how many commands you know. It is about asking the right questions and finding the correct answer.

5. ASK THE RIGHT QUESTIONS



- What changed?
- What is failing?
- When did it start?
- What evidence exists?
- What is the expected behaviour?
- What is different now?

6. START BROAD, THEN NARROW



- First check the big picture (system, network, resources, services).
- Then move to the specific area.
- Use logs, metrics and commands to narrow down.
- Keep following the evidence until you find the real cause.

2. SYMPTOMS vs ROOT CAUSES

SYMPTOMS

- Error messages
- Application not working
- Slow performance
- Service not starting
- Cannot connect
- High CPU or memory
- Disk full

A symptom tells you **WHAT** is happening.

ROOT CAUSES

- Misconfiguration
- Permission issues
- Missing files or packages
- Service failures
- Network or DNS problems
- Resource exhaustion
- Hardware or disk issues

A root cause tells you **WHY** it is happening.

4. THE TROUBLESHOOTING WORKFLOW



JUNIOR ENGINEER TIP

Never make the first error message automatically become your root cause.

The first error is a clue, not always the real problem. Investigate further, check the full context, and confirm before you fix.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOW WHERE YOU ARE BEFORE TROUBLESHOOTING

CHECK YOUR ENVIRONMENT • CONFIRM YOUR LOCATION • AVOID COSTLY MISTAKES

1. WHY THIS MATTERS



- Before you troubleshoot, you must know where you are, who you are, and what system you are working on.
- Running commands on the wrong server or in the wrong directory can lead to confusion, wasted time, or even accidental changes.
- Always confirm your environment first.

2. FILESYSTEM LOCATION



- Linux uses a file system organized in a tree structure, starting from / (root).
- Your current directory is your working location.
- You can use absolute or relative paths to reference files and directories.

3. ABSOLUTE vs RELATIVE PATHS

- Absolute path:** Full path from the root directory (/).
Example: `/home/user/projects/app`
- Relative path:** Path relative to your current directory.
Example: `./app` or `../logs`
- Use absolute paths in scripts and automation to avoid confusion.

4. USEFUL COMMANDS

Command	What It Does
<code>pwd</code>	Shows your current directory
<code>ls -la</code>	Lists files and directories in long format (including hidden files)
<code>cd <directory></code>	Changes to the specified directory
<code>whoami</code>	Shows the current user
<code>hostname</code>	Shows the server hostname
<code>uname -a</code>	Shows kernel and system information

5. EXAMPLE OUTPUT

```
[user@linux ~]$ pwd
/home/user
[user@linux ~]$ whoami
user
[user@linux ~]$ hostname
linux-server-01
[user@linux ~]$ uname -a
Linux linux-server-01 5.15.0-76-generic
x86_64 GNU/Linux
```

6. COMMON FAILURE



- Troubleshooting the correct file on the wrong server.
- Running commands from the wrong directory.
- Editing the wrong configuration file.
- Making changes as the wrong user.
- Assuming you are on the production server when you are on a test server.

Always verify your user, host, and current directory before investigating further.

7. PRACTICE: IDENTIFY YOUR ENVIRONMENT



- Run `pwd` → What is your current directory?
- Run `whoami` → Which user are you?
- Run `hostname` → What is the server name?

- Run `uname -a` → What kernel and OS are you using?
- Run `ls -la` → What files and directories are here?
- Navigate to another directory and confirm your location.
- Practice using both absolute and relative paths.

Do this before making any changes. Know where you are, then troubleshoot.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

"NO SUCH FILE OR DIRECTORY"

UNDERSTAND THE ERROR. FIND THE REAL CAUSE. APPLY THE RIGHT FIX.

1. WHAT THIS ERROR MEANS

```
[user@linux ~]$ cat /home/user/file.txt
cat: /home/user/file.txt:
No such file or directory
```

- This error means Linux cannot find the file or directory at the specified path.
- It does not always mean the file is deleted.
- It can be caused by a wrong path, a renamed file, a broken symlink, or a typo.

3. USEFUL COMMANDS

Command	What It Does
ls	Lists files and directories
find	Searches for files and directories
stat	Shows detailed information about a file or directory
readlink	Shows the target of a symbolic link
realpath	Shows the absolute path of a file

5. EXAMPLES

```
[user@linux ~]$ ls /home/user/file.txt
ls: cannot access '/home/user/file.txt':
No such file or directory

[user@linux ~]$ find /home -name file.txt
/home/user/documents/file.txt

[user@linux ~]$ readlink myapp
/opt/myapp/releases/v1 (No such file or directory)

[user@linux ~]$ realpath myapp
realpath: myapp: No such file or directory
```

7. PRACTICE



- Run **pwd** → Confirm your current directory.
- Run **ls -la** → Check what files exist.
- Use **find** to search for a file (e.g. **find / -name file.txt 2>/dev/null**).
- Check a symlink with **readlink** and validate with **realpath**.

2. COMMON CAUSES



Wrong paths
You typed the wrong location.



Renamed files
The file exists but with a different name.



Deleted files
The file was deleted.



Broken symbolic links
The link points to a file that no longer exists.



Case sensitivity
Linux is case-sensitive. File.txt and file.txt are different.

4. TROUBLESHOOTING FLOW

1

Verify the path
Check if you typed the correct path.

2

Inspect the parent directory
List the parent directory to see what exists.

3

Search for the file
Use **find** to search the filesystem.

4

Check if it is a symlink
Use **readlink** or **stat** to verify the link target.

6. BEGINNER MISTAKE



Assuming the file is deleted without verifying the path.

- You might be in the wrong directory.
- You might have a typo in the filename.
- The file might be in a different location.
- It might be a broken symbolic link.
- Linux is case-sensitive.
- Always investigate before assuming.



JUNIOR ENGINEER TIP

Never make the first error message automatically become your root cause. Investigate, check the full context, and verify the path, directory and links before you fix.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

"PERMISSION DENIED"

UNDERSTAND LINUX PERMISSIONS. FIND THE CAUSE. APPLY THE RIGHT FIX.

1. WHAT DOES THIS ERROR MEAN?



"Permission denied" means your user does not have the required permission to access a file, directory, or execute a command.

- This can happen with files, directories, commands, scripts, services, or SSH keys.

3. USEFUL COMMANDS

Command	What It Does
<code>ls -l</code>	Shows file permissions, owner and group
<code>id</code>	Shows your user and group information
<code>namei -l <path></code>	Shows permissions for each directory in the path
<code>chmod <mode> <file></code>	Changes file or directory permissions
<code>chown <user:group> <file></code>	Changes file or directory ownership

5. WHY chmod 777 IS NOT A GOOD FIX



- `chmod 777` gives everyone full access (read, write, execute).
- It removes security and can expose sensitive files.
- It may allow accidental or malicious changes.
- It does not fix the real problem (wrong owner, wrong group, or incorrect permission).
- Use the minimum permission required.

7. PRACTICE



- Run `ls -l` on a file and identify the owner, group and permissions.
- Use `id` to check your current user and groups.
- Use `namei -l /path/to/file` to inspect the full path.
- Try changing permissions with `chmod` (on a test file).
- Fix SSH key permissions to `600`.

2. LINUX USERS, GROUPS AND PERMISSIONS



User
(Owner)
e.g. user1



Group
e.g. devops



Others
e.g. everyone else

Each file and directory has permissions for:

Owner | Group | Others

Permissions:

r = read

View content
(ls, cat)

w = write

Modify content
(edit, delete)

x = execute

Run file or
enter directory

4. EXAMPLE: CHECK PERMISSIONS

```
[user@linux ~]$ ls -l /var/www/html
total 4
-rw-r--r-- 1 root root 1234 Jan 10 12:00 index.html
```

File type
(- = file,
d = directory)

Owner
permissions
(rw-)

Group
permissions
(r--)

Others
permissions
(r--)

6. SSH PRIVATE-KEY PERMISSIONS (REAL EXAMPLE)

SSH private keys must be protected. If permissions are too open, SSH will refuse to use the key.

```
[user@linux ~]$ ls -l ~/.ssh/id_rsa
-rw-r--r-- 1 user user 1675 Jan 10 10:00 ~/.ssh/id_rsa
[user@linux ~]$ chmod 600 ~/.ssh/id_rsa
[user@linux ~]$ ls -l ~/.ssh/id_rsa
-rw----- 1 user user 1675 Jan 10 10:00 ~/.ssh/id_rsa
```

Recommended permissions:

- Private key: `chmod 600 ~/.ssh/id_rsa`
- Directory: `chmod 700 ~/.ssh`

8. SECURITY REMINDER



Fix the specific permission problem, not every permission.

- Use least privilege.
- Understand what the permission controls.
- Do not use `chmod 777` as a shortcut.
- Keep sensitive files protected.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

"COMMAND NOT FOUND"

UNDERSTAND WHY IT HAPPENS. FIND THE CAUSE. APPLY THE RIGHT FIX.

1. WHAT DOES THIS ERROR MEAN?



"command not found" means Linux cannot find an executable file with that name in your PATH, or the command does not exist.

- The command might not be installed.
- It might be installed but not in your PATH.
- You might have a typo in the command name.
- You might not have execute permission.

2. HOW LINUX FINDS COMMANDS

You type a command (e.g. nginx)



Linux searches directories in your PATH (in order)



If it finds an executable file, it runs it.

Executable files

- Programs are executable files stored in directories such as `/usr/bin`, `/usr/local/bin`, `/bin`.
- The PATH environment variable tells Linux where to look for commands.

3. USEFUL COMMANDS

Command	What It Does
<code>command -v <cmd></code>	Shows the full path if the command exists (or nothing if not found).
<code>which <cmd></code>	Shows the path to the command (if available).
<code>type <cmd></code>	Shows how the command will be interpreted (alias, builtin or file).
<code>echo \$PATH</code>	Shows the directories Linux searches for commands.

4. EXAMPLE OUTPUT

```
[user@linux ~]$ command -v nginx
/usr/sbin/nginx
[user@linux ~]$ which nginx
/usr/sbin/nginx
[user@linux ~]$ type nginx
nginx is /usr/sbin/nginx
[user@linux ~]$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
```

5. COMMON CAUSES



Package missing

- The tool is not installed.
- Example: nginx, docker, terraform.



PATH problem

- Tool is installed but not in PATH.
- Common in custom installations like `/opt/tools/bin`.



Typo in command

- Misspelled command name.
- Linux commands are case-sensitive.



Execute permission

- File exists but is not executable.
- Fix with `chmod +x <file>`.

6. REAL EXAMPLE: WORKS INTERACTIVELY BUT FAILS IN AN AUTOMATION JOB

A tool works when you run it in your terminal, but fails in a script, CI/CD job, or cron job because the PATH is different.

```
# Works in your interactive shell
[user@linux ~]$ terraform --version
Terraform v1.6.0
# Fails in a script or cron job
$ terraform --version
bash: terraform: command not found
```

Why this happens?

- Interactive shells load your full PATH from `~/.bashrc` or `~/.zshrc`.
- Non-interactive environments (scripts, cron, CI/CD) may have a limited PATH.
- Solution: Use the full path (e.g. `/usr/local/bin/terraform`) or update the PATH in the script.

7. PRACTICE



- 1 Run `command -v <cmd>` for a tool you use.
- 2 Check your \$PATH and identify the directories.
- 3 Try a mistyped command and observe the error.
- 4 Find a tool installed in a non-standard location (e.g. `/opt`) and run it using the full path.
- 5 Create a simple script and compare PATH with your terminal.

8. KEY TAKEAWAYS



- ✓ Linux finds commands by searching the directories in \$PATH.
- ✓ The command might be missing, in the wrong location, misspelled, or not executable.
- ✓ Always verify with `command -v`, `which`, `type`, and `echo $PATH`.
- ✓ In scripts and automation, use full paths or set the PATH.
- ✓ Do not assume the command is missing until you investigate.

PROCESSES, PIDs, AND "MY APPLICATION IS NOT RUNNING"

FIND • UNDERSTAND • DIAGNOSE • TAKE THE RIGHT ACTION

1. WHAT IS A LINUX PROCESS?



Program



Process (PID)

- A process is a running program.
- It is an instance of a program in memory
- Every process has a unique Process ID (PID).
- Processes use system resources (CPU, memory, files, network).
- When a program is running, it becomes a process.

2. KEY CONCEPTS

Concept	Explanation
Process ID (PID)	Unique number assigned to each process (e.g. 1234).
Parent process (PPID)	The process that started another process.
Foreground process	Runs in the terminal and uses your input (e.g. top).
Background process	Runs without using your terminal (e.g. a service, script with &).
Daemon process	A background process that runs as a service (e.g. nginx, sshd).

3. USEFUL COMMANDS

Command	What It Does
<code>ps aux</code>	Shows all running processes with details.
<code>pgrep <name></code>	Finds the PID of a process by name.
<code>top</code>	Shows real-time running processes and resource usage.
<code>kill <PID></code>	Sends a signal to a process (e.g. stop).

4. EXAMPLE OUTPUT

```
[user@linux ~]$ ps aux | head -5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0  0.1  1660  1232 ?        Ss   10:00   0:02 /sbin/init
root      234   0.0  0.1  4568  2340 ?        Ss   10:00   0:00 /usr/sbin/sshd
user     1234   0.0  0.5  9876  6544 ?        S    10:05   0:01 nginx: master
user     1235   0.0  0.3  8764  4320 ?        S    10:05   0:00 nginx: worker

[user@linux ~]$ pgrep nginx
1234
1235
```

Multiple PIDs mean the application is running (master + workers).

5. PROCESS STATES (COMMON)

State	Meaning
R	Running (using CPU)
S	Sleeping (waiting for an event)
D	Uninterruptible sleep (usually I/O)
Z	Zombie (process finished but not cleaned up)
T	Stopped (e.g. by a signal)

6. PROCESS RUNNING vs APPLICATION HEALTHY



Process running

- Only means the process exists in the system.
- It does not always mean the application is working correctly.



Application healthy

- Means the application is running and working as expected (responding, no errors, serving traffic).
- You must check logs, metrics and functionality, not just the PID.

7. COMMON FAILURES



- Crashed process (exits unexpectedly).
- Stuck process (high CPU or not responding).
- Repeated restart (keeps crashing and restarting).
- Zombie process (not cleaned up).
- Process running but application returns errors.
- Wrong process or multiple old processes.

9. PRODUCTION AWARENESS



- Investigate before killing processes.
- Understand the impact (is it the right process?).
- Check logs and system state.
- Killing the wrong process can cause an outage.
- Use proper signals (e.g. SIGTERM) before SIGKILL.

8. PRACTICE



- 1 Run `ps aux` and identify key processes on your system.
- 2 Use `pgrep <name>` to find the PID of a process (e.g. nginx).
- 3 Open `top` and observe CPU and memory usage.
- 4 Start a simple process (e.g. `sleep 300`) and find its PID.
- 5 Try killing a test process with `kill <PID>` and observe the result.

10. KEY TAKEAWAYS



- ✓ A process is a running program with a unique PID.
- ✓ Use the right commands to investigate.
- ✓ A running process does not always mean a healthy application.
- ✓ Understand process states.
- ✓ Always investigate before killing any process.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

TROUBLESHOOTING SYSTEMD SERVICES

MANAGE • DIAGNOSE • FIX • KEEP SERVICES RUNNING

1. WHAT IS SYSTEMD?



- systemd is the init system and service manager used by most modern Linux distributions (e.g. Ubuntu, RHEL, Rocky).
- It starts, stops, and manages services (daemons) on your system.
- Services are defined in unit files (e.g. nginx.service) and controlled with the systemctl command.

3. USEFUL SYSTEMCTL COMMANDS

Command	What It Does
<code>systemctl status nginx</code>	Shows the current status and recent logs.
<code>systemctl start nginx</code>	Starts the service.
<code>systemctl restart nginx</code>	Restarts the service.
<code>systemctl enable nginx</code>	Enables the service to start at boot.
<code>systemctl is-active nginx</code>	Shows if the service is active.
<code>systemctl is-enabled nginx</code>	Shows if the service is enabled at boot.

4. EXAMPLE: CHECK NGINX SERVICE

```
[user@linux ~]$ systemctl status nginx
● nginx.service - A high performance web server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled)
   Active: active (running) since Thu 2024-01-10 10:00:00 UTC
     Main PID: 1234 (nginx)
        Tasks: 3 (limit: 4915)
       Memory: 5.2M
      CGroup: /system.slice/nginx.service
              └─1234 nginx: master process
                 └─1235 nginx: worker process
                 └─1236 nginx: worker process
```

7. PRACTICE



- 1 Check the status of nginx.
- 2 Stop, start, and restart the service.
- 3 Enable the service at boot.
- 4 View the last 50 log lines.
- 5 Find and fix a configuration error (e.g. wrong port).
- 6 Verify the service is running and enabled.

2. SERVICE STATES

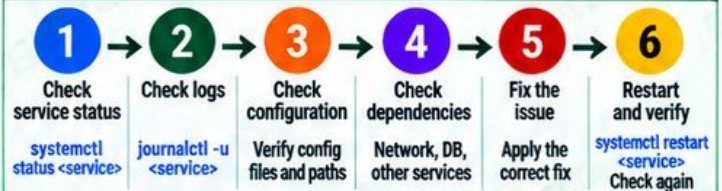
State	Meaning	Common Cause
active (running)	Service is running and healthy.	Working normally.
inactive (dead)	Service is not running.	Stopped manually or not started.
failed	Service failed to start or crashed.	Configuration error, missing dependency, permission issue, etc.
activating	Service is starting (up).	During startup.
reloading	Service is reloading its configuration.	Normal during reload.
deactivating	Service is stopping (down).	During shutdown.
activating (auto-restart)	Service keeps restarting.	Service failing repeatedly.

5. WHY A SERVICE CAN FAIL IMMEDIATELY



- Syntax error in the configuration file.
- Missing dependencies (e.g. network, database).
- Port already in use.
- Incorrect user or permission.
- Missing environment variables.
- Invalid paths to files, certificates, or directories.
- Resource limits (memory, files, etc.).
- Service exits immediately due to an application error.

6. TROUBLESHOOTING FLOW



8. KEY TAKEAWAYS



- ✓ Use systemctl to manage services.
- ✓ Always check logs when a service fails.
- ✓ Understand the common reasons for failure.
- ✓ Follow a structured troubleshooting flow.
- ✓ Fix the real issue, not just restart repeatedly.
- ✓ Keep services enabled if they should start at boot.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

READING LOGS WITHOUT GETTING LOST

FIND EVIDENCE • UNDERSTAND ERRORS • FIX PROBLEMS

1. WHY LOGS ARE IMPORTANT



- Logs are evidence of what happened on your system.
- They show you events, errors, and useful details for troubleshooting.
- Logs help you understand the real cause, not just the symptom.
- Always check logs when something is not working.

No guessing. Check the logs.

2. COMMON LOG LOCATIONS

**/var/log**

- Most system and application logs are stored in **/var/log**.
- Different services and applications have their own log files.
- Examples: **/var/log/syslog**, **/var/log/auth.log**, **/var/log/nginx/**, **/var/log/messages**.
- On modern Linux systems, many logs are also available through the systemd **journal**.

3. USEFUL LOG COMMANDS

Command	What It Does
journalctl	View logs from the systemd journal.
journalctl -u nginx	View logs for a specific service (nginx).
journalctl -xe	Show recent logs with more details (including errors).
tail -f <file>	Follow a log file in real time.
less <file>	View a log file page by page.
grep <pattern> <file>	Search for specific text in a log file.

4. EXAMPLE COMMANDS

```
[user@linux ~]$ cd /var/log
[user@linux log]$ ls
auth.log      nginx/        syslog
kern.log      messages      ufw.log
nginx/        secure        user.log
[user@linux log]$ journalctl -u nginx
[user@linux log]$ journalctl -xe
[user@linux log]$ tail -f /var/log/syslog
[user@linux log]$ less /var/log/nginx/error.log
[user@linux log]$ grep error /var/log/nginx/error.log
```

5. UNDERSTANDING LOG ENTRIES

Timestamp Hostname Service Severity Message

Jan 10 10:15:23 server1 nginx[1234]: [error] failed to open configuration file /etc/nginx/nginx.conf: No such file or directory

Part	Meaning
Timestamp	When the event happened.
Hostname	Which server generated the log.
Service / Process	Which application or component.
Severity	Level of the log (e.g. info, warning, error, critical).
Message	Details about what happened.

6. COMMON SEVERITY LEVELS

Level	Meaning
emerg	System is unusable (very serious).
alert	Immediate action required.
crit	Critical condition.
err	Error (something failed).
warning	Warning (potential issue).
notice	Normal but significant.
info	Informational message.
debug	Detailed debugging information.

7. PRACTICE



- 1 View recent system logs with **journalctl**.
- 2 Check logs for a specific service (e.g. **nginx**).
- 3 Follow a log file in real time with **tail -f**.
- 4 Search for errors using **grep**.
- 5 Identify the timestamp, severity and what happened.
- 6 Find the first meaningful failure and investigate it.

8. TROUBLESHOOTING TIP

**Find the first meaningful failure, not simply the last line.**

- ✓ Look for the first error, not just the last line.
- ✓ Read surrounding context (lines before and after).
- ✓ Understand what changed around the time of the error.
- ✓ Use timestamps to correlate with other logs and events.
- ✓ Don't ignore warnings; they can explain later failures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

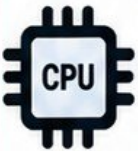


@veriqta

CPU PROBLEMS AND SLOW LINUX SERVERS

UNDERSTAND • DIAGNOSE • FIND THE CAUSE • TAKE THE RIGHT ACTION

1. WHAT IS CPU UTILIZATION?



- CPU utilization shows how much of the CPU's processing power is being used.
- 100% means the CPU is fully busy.
- High CPU usage can make your server slow or unresponsive.
- Always identify which process is using the CPU before taking action.

High CPU usage is a symptom.
Find the process and the cause.

2. LOAD AVERAGE EXPLAINED

- Load average shows the number of processes waiting for CPU (or I/O) over the last 1, 5 and 15 minutes.
- It is not a percentage.
- A load of 1.0 means one process is waiting (or running).
- Compare the load with the number of CPU cores.

Load Value	Meaning
< number of cores	Normal
= number of cores	Busy
> number of cores	High load (potential problem)

3. CPU-BOUND PROCESSES

- A CPU-bound process uses a lot of CPU and does not spend much time waiting for I/O (disk, network).
- Examples: compression, data processing, infinite loops, high-traffic applications.
- These processes can make the system slow for other users and services.

Find CPU-bound processes
and decide whether to optimize,
restart or stop them.

4. USEFUL COMMANDS

Command	What It Does
<code>uptime</code>	Shows system uptime and load average.
<code>top</code>	Shows real-time CPU usage per process.
<code>ps aux</code>	Shows running processes with details.
<code>lscpu</code>	Shows CPU architecture and number of cores.
<code>mpstat</code>	Shows CPU usage per core (from sysstat).
<code>pidstat</code>	Shows CPU usage by process (from sysstat).

5. EXAMPLE: CHECK CPU AND LOAD

```
[user@linux ~]$ uptime
10:24:36 up 5 days, 3:12, 2 users, load average: 4.25, 3.10, 1.85

[user@linux ~]$ top - 5
  PID USER   %CPU %MEM    COMMAND
 1234 appuser 320.5   1.2    java
 5678 root    45.3   0.5    python
 9012 nginx   12.8   0.3    nginx
 3456 root     8.1   0.1    kworker/u16:1
 7890 mysql   5.4   2.1    mysqld
```

Load is high (> number of cores)

Process using high CPU

6. HIGH CPU vs HIGH LOAD

High CPU Usage

- CPU is busy (high utilization).
- One or more processes are using a lot of CPU.
- Can happen on a single core or multiple cores.
- Check with `top`, `ps` or `htop`.

High Load Average

- Many processes are waiting for CPU (or I/O).
- Does not always mean CPU is at 100%.
- Can be caused by CPU, disk I/O or other resource limits.
- Check with `uptime`, `top`, `vmstat` and `iostat`.

7. SCENARIO: APPLICATION IS EXTREMELY SLOW



- Users report the application is very slow.
- Server CPU usage is high and load average is high.
- You need to find which process is causing the problem.

Troubleshooting Steps:

- 1 Check system load with `uptime`.
- 2 Identify high CPU processes with `top` or `ps aux`.
- 3 Investigate the process (is it expected?).
- 4 Check application logs (if needed).
- 5 Decide the right action (optimize, restart, stop).
- 6 Monitor after the fix.

8. PRACTICE



- 1 Run `uptime` and note the load average.
- 2 Use `top` to find the process with highest CPU.
- 3 Use `ps aux | sort -k3 -nr | head -10` to list top CPU processes.
- 4 Check CPU information with `lscpu`.
- 5 Find and investigate a high CPU process on your system.
- 6 Document what you found and the action you took.

9. KEY TAKEAWAYS



- ✓ CPU utilization shows how busy the CPU is.
- ✓ Load average shows the number of processes waiting.
- ✓ High CPU and high load can make the server slow.
- ✓ Always identify the process before taking action.
- ✓ Use the right tools: `uptime`, `top`, `ps`, `lscpu`.
- ✓ Investigate the cause, don't just kill the process.
- ✓ Monitor after making changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

MEMORY PROBLEMS AND THE OOM KILLER

UNDERSTAND • INVESTIGATE • FIND THE CAUSE • APPLY THE RIGHT FIX

1. LINUX MEMORY BASICS



- RAM (physical memory) is used by running processes, the OS, and caches.
- Available memory is what can be used for new processes.
- Buffers and cache are memory used to improve performance (they can be released when needed).
- Swap is disk space used as extra memory when RAM is not enough (slower than RAM).

Linux will use available memory for cache, and that is normal. A low "free" value does not automatically mean a problem.

2. USEFUL COMMANDS

Command	What It Does
<code>free -h</code>	Shows memory usage in a human-readable format.
<code>top</code>	Shows real-time memory usage by processes.
<code>ps aux</code>	Shows all running processes with memory usage.
<code>cat /proc/meminfo</code>	Shows detailed memory information from the kernel.
<code>vmstat 1</code>	Shows memory, CPU, swap and I/O activity.
<code>swapon --show</code>	Shows configured swap space.
<code>dmesg grep -i oom</code>	Searches kernel logs for OOM events.

3. KEY MEMORY TERMS

Term	Meaning
Total	Total physical RAM.
Used	Memory currently in use (by processes, buffers, cache).
Free	Memory not used (often small).
Buffers	Memory used for block devices (e.g. disk I/O).
Cache	Memory used to cache files (if needed, it can be released).
Available	Estimated memory available for new processes.
Swap	Disk space used as extra memory.

4. EXAMPLE: CHECK MEMORY USAGE

```
[user@linux ~]$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.6G         6.1G         300M         200M         1.2G         1.1G
Swap:         2.0G         512M         1.5G
```

```
[user@linux ~]$ cat /proc/meminfo | head -5
MemTotal:        8147488 kB
MemFree:          314572 kB
MemAvailable:    1187656 kB
Buffers:          204800 kB
Cached:          1048576 kB
```

Use these commands to understand real memory usage.

5. WHAT IS OOM AND THE OOM KILLER?

- OOM (Out Of Memory) happens when the system does not have enough memory to allocate to a new process.
- The Linux OOM killer automatically terminates one or more processes to free memory.
- It usually kills the process using the most memory or the one with the highest OOM score.
- You may see "Killed" in logs or the application suddenly stops without a clear error.



The OOM killer is a last resort to keep the system running. It does not mean the kernel is broken.

6. HOW TO INVESTIGATE AN OOM EVENT

- 1 Check system logs: `dmesg | grep -i oom` or `journalctl -k | grep -i oom`
- 2 Identify the killed process (name, PID, memory usage).
- 3 Check what the application was doing (logs, recent changes).
- 4 Look at memory usage over time (`top`, `vmstat`, monitoring tools).
- 5 Verify if there are memory leaks in the application.
- 6 Consider increasing memory, optimizing the application, or setting limits (e.g. in containers).

7. EXAMPLE: OOM KILLER LOG

```
[user@linux ~]$ dmesg | grep -i oom
[12345.678901] Out of memory: Killed process 4321 (java)
[12345.678905] oom-kill:constraint=CONSTRAINT_NONE,
nodemask=(null),cpuset=/,mems_allowed=0-3,
oom_score_adj=0
[12345.678910] Killed process 4321 (java) total-vm:2048000kB,
anon-rss:1500000kB, file-rss:0kB, shmem-rss:0kB, UID:1000
```

Kernel log shows which process was killed and why.

8. PRODUCTION AWARENESS



- Restarting the application may make it work again, but it does not fix the underlying memory problem.
- Repeated OOM kills can cause downtime and data loss.
- Monitor memory usage and set proper alerts.
- Investigate and fix the root cause (e.g. memory leak, high load, wrong configuration) instead of only increasing RAM.

9. KEY TAKEAWAYS



- ✓ Understand RAM, available memory, buffers/cache and swap.
- ✓ Use the right commands to check memory usage.
- ✓ Look for OOM events in system logs.
- ✓ Identify which process was killed and why.
- ✓ Fix the root cause, not just restart the application.
- ✓ Monitor memory to prevent future issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

"NO SPACE LEFT ON DEVICE" DISK USAGE TROUBLESHOOTING

FIND THE CAUSE • UNDERSTAND • FIX • PREVENT

1. WHAT DOES THIS ERROR MEAN?



- This error means the filesystem you are writing to is full.
- It can also happen when inodes are exhausted even if there is free disk space.
- Your application, logs, or system processes cannot create or write new files.
- The server may still appear operational, but new data cannot be saved.

No space left on device = the filesystem cannot allocate more space (or inodes).

2. DISK CAPACITY vs FILESYSTEM USAGE

Disk Capacity

- Total size of the disk (e.g. 100 GB).
- Shown by tools like `lsblk`.
- Physical or virtual disk size.
- Can be larger than any individual filesystem.



Filesystem Usage

- How much of a partition or filesystem is used.
- Shown by `df -h`.
- Each filesystem can be full even if the overall disk still has space in another partition.

You can have free disk space on `/dev/sda` but still be full on `/` (root filesystem).

3. USEFUL COMMANDS

Command	What It Does
<code>df -h</code>	Shows filesystem disk usage in human-readable format.
<code>du -sh <directory></code>	Shows the total size of a directory.
<code>du -xhd1</code>	Shows size of each subdirectory (one level deep, same filesystem).
<code>lsblk</code>	Shows disk devices and partitions.
<code>df -i</code>	Shows inode usage (important for inode exhaustion).
<code>lsuf +L1</code>	Shows deleted files that are still open and consuming space.

4. EXAMPLE OUTPUT

```
[user@linux ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        50G   50G   0 100% /
/dev/root       1.0G   200M   800M  20% /boot
tmpfs           2.0G    12M   1.9G   1% /dev/shm

[user@linux ~]$ df -i
Filesystem      Inodes   IUsed   IFree IUse% Mounted on
/dev/root     3276800 3276800    0 100% /
/dev/sda1     655360  12345  643015    2% /boot

[user@linux ~]$ lsuf +L1 | head -5
COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE/OFF  NLINK  NAME
nginx    1234  nginx  7w   REG   253,1    104857600  0      /var/log/nginx/access.log (deleted)
java     5678  app    8w   REG   253,1    2147483648  0      /app/app.log (deleted)
```

Root filesystem is full (100% used)

Inodes exhausted (100% used)

5. COMMON CAUSES

- Large logs**
 - Log files can grow very large (e.g. `/var/log`). Applications may not be rotating or compressing logs.
- Application data**
 - Databases, uploads, or temporary files can consume large amounts of space.
- Deleted files still open**
 - Files deleted but still held open by a running process continue to consume disk space.
- Inode exhaustion**
 - Too many files (even small files) can exhaust inodes. This stops new files from being created.

6. SCENARIO



Application cannot write files even though the server still appears operational.

- The server is up and running.
- Users can still access the application.
- But the application cannot write logs, upload files, or save data.
- Checking `df -h` shows the filesystem is full (100%).
- It could also be inode exhaustion, even if `df -h` shows free space.
- You need to investigate disk usage, large files, open deleted files, or inodes.

7. TROUBLESHOOTING STEPS

- 1 Check disk usage with `df -h` and `df -i`.
- 2 Find large directories with `du -xhd1 /`.
- 3 Identify large files (e.g. logs, core dumps, temp files).
- 4 Check for deleted files still open with `lsuf +L1`.
- 5 Investigate inode usage if you cannot create new files.
- 6 Clean up unnecessary files, rotate logs, or increase disk size.
- 7 Verify that the application can write files again.

8. KEY TAKEAWAYS



- ✓ "No space left on device" means the filesystem is full or inodes are exhausted.
- ✓ Use `df -h`, `du`, `lsblk`, `df -i` and `lsuf` to investigate.
- ✓ Large logs and application data are common causes.
- ✓ Deleted files can still consume space if open.
- ✓ Inode exhaustion can stop file creation even if disk space is available.
- ✓ Always find the root cause, not just delete random files.
- ✓ Implement log rotation and monitoring to prevent recurrence.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

FILESYSTEM AND MOUNT TROUBLESHOOTING

UNDERSTAND • DIAGNOSE • FIX • PREVENT

1. KEY CONCEPTS



- **Disk:** physical or virtual storage device (e.g. /dev/sda).
- **Partition:** a section of a disk (e.g. /dev/sda1).
- **Filesystem:** structure used to store files (e.g. ext4, xfs).
- **Mount point:** directory where a filesystem is attached (e.g. /, /var, /data).
- A disk can exist but its files appear unavailable if the filesystem is not mounted, is corrupted, or has a mount problem.

A disk existing does not mean its files are accessible.
The filesystem must be correctly mounted.

2. USEFUL COMMANDS

Command	What It Does
<code>lsblk</code>	Shows disks, partitions and mount points.
<code>findmnt</code>	Shows mounted filesystems in a tree view.
<code>mount</code>	Shows currently mounted filesystems.
<code>df -hT</code>	Shows disk usage and filesystem types.
<code>cat /etc/fstab</code>	Shows filesystems that automatically mount at boot.

3. EXAMPLE: CHECK DISKS AND MOUNTS

Check disks and partitions:

```
[user@linux ~]$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
sda   8:0    0 100G 0 disk 
├─sda1 8:1    0   1G 0 part /boot
├─sda2 8:2    0   99G 0 part /
└─sdb   8:16   0 200G 0 disk 
  ├─sdb1 8:17   0 200G 0 part /data
```

Check filesystem usage and type:

```
[user@linux ~]$ df -hT
Filesystem Type Size Used Avail Use% Mounted on
/dev/sda2 xfs 99G 28G 79G 21% /
/dev/sda1 xfs 1G 200M 800M 20% /boot
/dev/sdb1 ext4 200G 50G 150G 25% /data
```

4. UNDERSTANDING MOUNT STATES

Issue	What It Means
Mounted	Filesystem is attached and accessible.
Not mounted	Disk/partition exists but is not mounted, so files are not accessible.
Read-only	Filesystem is mounted but in read-only mode (often due to errors).
Failed mount	System could not mount the filesystem (e.g. wrong device, filesystem error, network issue).
Stale mount	Mount point exists but the underlying device is not available.

5. CHECK MOUNT INFORMATION

View mounted filesystems:

```
[user@linux ~]$ findmnt
TARGET SOURCE FSTYPE OPTIONS
/       /dev/sda2 xfs     rw,relatime
/boot   /dev/sda1 xfs     rw,relatime
/data   /dev/sdb1 ext4     rw,relatime
```

Check a specific mount point:

```
[user@linux ~]$ findmnt /data
TARGET SOURCE FSTYPE OPTIONS
/data   /dev/sdb1 ext4     rw,relatime
```

6. /etc/fstab EXAMPLE

```
# /etc/fstab: static file system info
# <device> <mount point> <type> <options> <dump> <pass>
/dev/sda2 / xfs defaults 0 1
/dev/sda1 /boot xfs defaults 0 2
/dev/sdb1 /data ext4 defaults 0 2
```



IMPORTANT

- Editing `/etc/fstab` incorrectly can prevent systems from booting.
- Always test changes before rebooting.
- Use correct device names, filesystem types and mount options.

7. COMMON PROBLEMS AND CAUSES

Problem	Common Causes
Disk exists but files not accessible	Filesystem not mounted, wrong mount point, fstab misconfiguration.
Read-only filesystem	Filesystem errors, unclean shutdown, reached error state.
Failed to mount	Wrong device name, wrong filesystem type, missing dependencies, network storage issue.
Permission denied	Incorrect mount options or permissions.
Device not found	Device name changed (e.g. /dev/sdb → /dev/sdc).

9. REAL-WORLD SCENARIO



Your application cannot write files to `/data` even though the server appears to be running.

- Application shows "No space left" or "Read-only file system".
- Users cannot upload files.
- The disk exists (`lsblk` shows the device), but it is not mounted, or it is mounted as read-only.
- Check `df -hT`, `findmnt`, and system logs to find the root cause.

8. PRACTICE: TRACE DEVICE → FS → MOUNT



- 1 List disks and partitions with `lsblk`.
- 2 Check which filesystems are mounted with `findmnt` or `mount`.
- 3 Verify disk usage with `df -hT`.
- 4 If a filesystem is missing, check `/etc/fstab`.
- 5 Check for errors in system logs: `journalctl -xe`.
- 6 Fix the issue (e.g. mount manually or update fstab).
- 7 Verify the filesystem is accessible.

10. KEY TAKEAWAYS



- ✓ Understand the relationship between disk, partition, filesystem and mount point.
- ✓ Use `lsblk`, `findmnt`, `mount`, and `df -hT` for troubleshooting.
- ✓ A disk can exist but its files can be unavailable.
- ✓ Read-only filesystems indicate a problem that needs investigation.
- ✓ Be careful when editing `/etc/fstab` (it can prevent the system from booting).
- ✓ Always test changes and verify mounts.
- ✓ Follow a clear troubleshooting flow from device to filesystem to mount point.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

NETWORK TROUBLESHOOTING FROM THE LINUX HOST

START LOCAL • FIND THE ISSUE • VERIFY THE PATH • FIX THE PROBLEM

1. START WITH THE LOCAL MACHINE



- Always start troubleshooting from the local machine.
- Check if your own server is correctly configured before looking at the remote server.
- Verify network interfaces, IP addresses, routes and name resolution.
- Many network problems are local configuration issues.

A working local network configuration is the foundation for everything else.

2. NETWORK INTERFACES AND IP ADDRESSES

- Network interfaces connect your server to the network.
- Each interface can have an IP address (IPv4 or IPv6).
- Common interface names: eth0, ens5, enp0s3, etc.

```
[user@linux ~]$ ip addr
1: lo: <LOOPBACK,UP> ...
   inet 127.0.0.1/8 scope host lo
   inet6 ::1/128 scope host
2: ens5: <BROADCAST,UP> ...
   inet 192.168.1.10/24 brd 192.168.1.255
   inet6 fe80::a00:27ff:fe4e:66a1/64
```

Check that your interface is UP and has the expected IP address.

3. USEFUL COMMANDS

Command	What It Does
<code>ip addr</code>	Shows network interfaces and IP addresses.
<code>ip link</code>	Shows and manages network interfaces.
<code>ip route</code>	Shows the routing table (default gateway, routes).
<code>ping <host></code>	Tests connectivity to a host (ICMP).
<code>tracpath <host></code>	Shows the network path to a destination.
<code>ss -tulnp</code>	Shows listening ports and services.

4. UNDERSTANDING IP ADDRESSES

Localhost
(127.0.0.1)

- Refers to your own machine.
- Used for local testing (e.g. `ping 127.0.0.1`).
- Not reachable from other machines.

Private IP
(10.x.x.x
172.16.x.x
192.168.x.x)

- Used inside private networks (e.g. office, cloud VPC, data center).
- Not directly reachable from the internet.
- Used for internal communication between servers.

Public IP

- Used on the internet.
- Reachable from anywhere (if allowed by firewall/security groups).
- Can be static or dynamic.

5. CHECK ROUTING

- The routing table tells your server where to send traffic.
- You need a correct default route to reach external networks.

```
[user@linux ~]$ ip route
default via 192.168.1.1
dev ens5
192.168.1.0/24 dev ens5
proto kernel scope link
src 192.168.1.10
```

Make sure you have the correct default gateway and routes.

6. TEST CONNECTIVITY

- Test local connectivity (localhost):

```
[user@linux ~]$ ping 127.0.0.1
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.026 ms
```

- Test connectivity to another server:

```
[user@linux ~]$ ping 192.168.1.20
64 bytes from 192.168.1.20: icmp_seq=1 ttl=64 time=0.345 ms
```

- Trace the network path:

```
[user@linux ~]$ tracpath 192.168.1.20
1: 192.168.1.1 0.324 ms
2: 192.168.1.20 0.681 ms
```

If ping works but the application still fails, check the specific service and port (e.g. HTTP, SSH).

7. SCENARIO: SERVER CANNOT REACH ANOTHER SERVER



Server A
(192.168.1.10)



Server B
(192.168.1.20)

- Symptoms: `ping` fails or application cannot connect.
- Possible causes:

- 1 Interface is down or wrong IP address.
- 2 No route to the destination.
- 3 Firewall or security group blocking traffic.
- 4 Destination server is down.
- 5 Service is not listening on the expected port.
- 6 Network ACL or intermediate device blocking traffic.

8. TROUBLESHOOTING FLOW



9. ADDITIONAL HELPFUL COMMANDS

Command	What It Does
<code>hostname -I</code>	Shows the server's IP addresses.
<code>cat /etc/resolv.conf</code>	Shows DNS configuration.
<code>nslookup <domain></code>	Tests DNS name resolution.
<code>dig <domain></code>	Detailed DNS query.
<code>ss -tulnp</code>	Shows listening ports and services.
<code>telnet <host> <port></code>	Tests if a specific port is open (e.g. <code>telnet 192.168.1.20 22</code>).

10. KEY TAKEAWAYS



- ✓ Always start from the local machine.
- ✓ Verify interface, IP address, and routing.
- ✓ Use ping and tracpath to test connectivity.
- ✓ Understand the difference between localhost, private IP, and public IP.
- ✓ If the network path works, check the specific service.
- ✓ Follow a systematic flow and check each layer.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

DNS TROUBLESHOOTING

UNDERSTAND • DIAGNOSE • SOLVE • KEEP SERVICES RUNNING

1. NAME vs IP ADDRESS

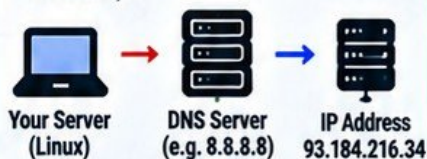


- A domain name (e.g. `www.example.com`) is easier for humans to remember.
- DNS (Domain Name System) translates the name to an IP address.
- Your application uses the IP address to connect to the server.

If DNS fails, the name cannot be resolved, even if the server is healthy and reachable.

2. DNS RESOLUTION EXPLAINED

- When you request a domain name, your system asks a DNS server for the corresponding IP address.
- The DNS server checks its cache or queries other DNS servers.
- If found, it returns the IP address.
- If not found, you get a DNS error (e.g. "name not found").



DNS is a critical dependency for almost every application.

3. IMPORTANT FILE

`/etc/resolv.conf`

- Contains the DNS servers your system uses for name resolution.
- Usually managed by your OS or network configuration (e.g. `systemd-resolved`, NetworkManager, DHCP).

```
[user@linux ~]$ cat /etc/resolv.conf
# Generated by NetworkManager
nameserver 8.8.8.8
nameserver 1.1.1.1
search example.com
```

If the DNS servers here are incorrect, names will not resolve.

4. USEFUL COMMANDS

Command	What It Does
<code>getent hosts <name></code>	Uses system NSS to resolve a name (same method as most applications).
<code>dig <name></code>	Detailed DNS query and response information.
<code>nslookup <name></code>	Simple DNS query tool.
<code>cat /etc/resolv.conf</code>	Shows configured DNS servers.
<code>host <name></code>	Another DNS query tool.
<code>ping <name></code>	Tests name resolution and connectivity.

5. EXAMPLE COMMANDS

```
[user@linux ~]$ getent hosts www.google.com
142.250.190.4 www.google.com
```

Name resolved to an IP address

```
[user@linux ~]$ dig www.google.com
;; ANSWER SECTION:
www.google.com. 300 IN A 142.250.190.4
```

```
[user@linux ~]$ nslookup www.google.com
Server:      8.8.8.8
Address:     8.8.8.8#53
Name:        www.google.com
Address:     142.250.190.4
```

All commands should return a valid IP address if DNS is working.

6. SCENARIO: IP WORKS BUT HOSTNAME DOES NOT

- You can ping the server using its IP address, but pinging the hostname fails.
- This means the network is working, but DNS resolution is failing.

```
[user@linux ~]$ ping 192.168.1.20
64 bytes from 192.168.1.20: icmp_seq=1 ttl=64 time=0.345 ms

[user@linux ~]$ ping app.example.com
ping: app.example.com: Name or service not known
```

This is a DNS problem, not a general network problem.

7. SEPARATE DNS FROM NETWORK FAILURES

- ✓ 1. Test IP connectivity
`ping <IP-address>`
- ✓ 2. Test DNS resolution
`getent hosts <name>` or `dig <name>`
- ✓ 3. If IP works but name fails
It is a DNS issue.
- ✓ 4. If both IP and name fail
It is a network issue (e.g. firewall, routing).

8. DEVOPS CONNECTION



- Broken DNS can make healthy applications appear unavailable.
- Microservices, databases, and APIs often rely on DNS.
- A DNS issue can cause deployment failures, service outages, and intermittent errors.
- Always include DNS checks in your troubleshooting process.

In production, DNS is part of your infrastructure. Keep it monitored and highly available.

9. COMMON DNS ERRORS



- Name or service not known (cannot resolve name)
- Temporary failure in name resolution
- No answer / NXDOMAIN (domain does not exist)
- Timeout (DNS server not reachable)

10. KEY TAKEAWAYS



- ✓ Understand the difference between a name and an IP address.
- ✓ Use `/etc/resolv.conf`, `getent`, `dig`, and `nslookup` to troubleshoot DNS.
- ✓ If IP works but hostname does not, it is a DNS issue.
- ✓ Always separate DNS failures from general network failures.
- ✓ Broken DNS can make healthy applications appear unavailable.
- ✓ Monitor DNS and use reliable, redundant DNS servers.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

"CONNECTION REFUSED" AND PORT TROUBLESHOOTING

DIAGNOSE • UNDERSTAND • FIX • KEEP SERVICES RUNNING

1. WHAT IS A NETWORK PORT?

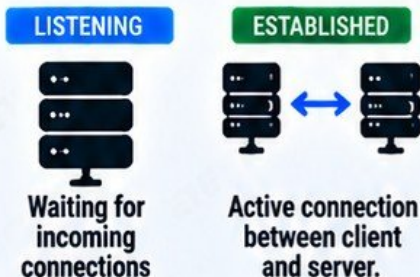
- A port is a number (0-65535) that identifies a specific service on a server.
- It allows multiple services to run on the same server at the same time.
- Common ports: 22 (SSH), 80 (HTTP), 443 (HTTPS), 3306 (MySQL), 6379 (Redis).



A service must be running and listening on a port for other machines to connect to it.

2. LISTENING vs ESTABLISHED

- Listening:** the server is waiting for incoming connections.
- Established:** a connection is already active between the client and server.



3. TCP BASICS

- TCP (Transmission Control Protocol) is the most common network protocol in Linux.
- It provides reliable, ordered, and error-checked communication between clients and servers.
- A TCP connection uses a source port (client) and a destination port (server).



4. USEFUL COMMANDS

Command	What It Does
<code>ss -ltnp</code>	Shows listening ports and the process using them.
<code>curl <host>:<port></code>	Tests connectivity to a service.
<code>nc -zv <host> <port></code>	Tests if a port is open.
<code>lsof -i</code>	Shows open network connections and the process using them.
<code>telnet <host> <port></code>	Tests connectivity (if installed).

5. EXAMPLE COMMAND OUTPUT

```
# Check listening ports
[user@linux ~]$ ss -ltnp
State Recv-Q Send-Q Local Address:Port Process
LISTEN 0 128 0.0.0.0:22 users:("sshd")
LISTEN 0 128 127.0.0.1:3306 users:("mysqld")
LISTEN 0 128 0.0.0.0:80 users:("nginx")
```

```
# Test with curl
[user@linux ~]$ curl http://localhost:80
<html>
<h1>Welcome to Nginx!</h1>
</html>
```

```
# Test with nc
[user@linux ~]$ nc -zv 192.168.1.20 80
Connection to 192.168.1.20 80 port [tcp/http] succeeded!
```

6. COMMON ERRORS

CONNECTION REFUSED

- Service is not running.
- Nothing is listening on the port.
- Firewall is actively rejecting the connection.

TIMEOUT

- No response from the server.
- Server is unreachable.
- Firewall or network issue blocking traffic.

OTHER COMMON CAUSES

- Application listening only on 127.0.0.1 (localhost).
- Wrong port.
- Service bound to the wrong interface (e.g. IPv6 only).
- Firewall/security group blocking the port.

7. APPLICATION LISTENING ONLY ON 127.0.0.1

- If a service listens on 127.0.0.1, only the local machine can access it.
- Remote machines will get "connection refused" or timeout.
- Check the listening address with `ss -ltnp` or `lsof -i`.

```
[user@linux ~]$ ss -ltnp | grep 8080
LISTEN 0 128 127.0.0.1:8080 users:("java")
```

Change the application configuration to listen on 0.0.0.0 if it should be accessible from other machines.

8. TROUBLESHOOTING FLOW

- 1 Check the process (is the service running?).
- 2 Check the listening socket (`ss -ltnp` or `lsof -i`).
- 3 Verify the address (0.0.0.0 vs 127.0.0.1).
- 4 Verify the port (is it the correct port?).
- 5 Test connectivity (`curl`, `nc`, `telnet`).

9. KEY TAKEAWAYS

- ✓ A port represents a specific service on a server.
- ✓ Use `ss -ltnp`, `curl`, `nc`, and `lsof -i` for troubleshooting.
- ✓ "Connection refused" usually means nothing is listening.
- ✓ A timeout usually means the server is unreachable.
- ✓ Always check the listening address (127.0.0.1 vs 0.0.0.0).
- ✓ Follow a systematic flow: process → listening socket → address → port → connectivity.
- ✓ Broken connectivity can make a healthy application appear unavailable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

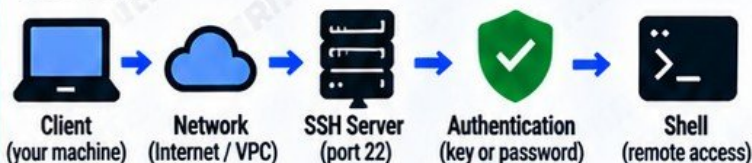
SSH TROUBLESHOOTING

DIAGNOSE • UNDERSTAND • FIX • SECURE • STAY CONNECTED

FROM CONNECTION TO SHELL

1. SSH CONNECTION PATH

A successful SSH connection goes through these steps:



If any step fails, the connection will not be established.
Troubleshoot each layer.

2. USEFUL COMMANDS

Command	What It Does
<code>ssh -v user@host</code>	Verbose output. Shows detailed connection and authentication steps.
<code>ss -lntp grep ssh</code>	Checks if SSH is listening and shows the process.
<code>systemctl status ssh</code>	Shows the status of the SSH service.
<code>journalctl -u ssh</code>	Shows SSH logs (useful for errors).
<code>cat ~/.ssh/authorized_keys</code>	Shows the authorized public keys.
<code>ls -l ~/.ssh/</code>	Checks SSH directory and file permissions.

3. CHECK IF SSH IS RUNNING

Verify the SSH service is running on the server.

```
[user@linux ~]$ systemctl status ssh
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service)
   Active: active (running) since Tue 2024-08-27 10:15:00
     Docs: man:sshd(8)
   Main PID: 1234 (sshd)
```

If SSH is not running, start it and check the logs.

4. CHECK LISTENING PORT

Confirm SSH is listening on port 22 (or a custom port).

```
[user@linux ~]$ ss -lntp | grep ssh
State Recv-Q Send-Q Local Address:Port
LISTEN 0      128   0.0.0.0:22
LISTEN 0      128   ::::22
```

If you do not see port 22 (or your custom port), SSH is not listening.

5. SSH DIRECTORY AND KEYS

Your SSH keys are stored in `~/.ssh/`

```
[user@linux ~]$ ls -l ~/.ssh/
total 12
-rw----- 1 user user 2602 id_rsa
-rw-r--r-- 1 user user 571 id_rsa.pub
-rw-r--r-- 1 user user 489 authorized_keys
-rw-r--r-- 1 user user 231 known_hosts
```

Verify that `authorized_keys` contains your public key and permissions are correct.

6. PRIVATE-KEY PERMISSIONS

SSH will reject a private key if permissions are too open.

```
[user@linux ~]$ ls -l ~/.ssh/id_rsa
-rw----- 1 user user 2602 Aug 27 10:00
~/.ssh/id_rsa
[user@linux ~]$ chmod 600 ~/.ssh/id_rsa
[user@linux ~]$ ls -l ~/.ssh/id_rsa
-rw----- 1 user user 2602 Aug 27 10:00
~/.ssh/id_rsa
```

Private key files should usually be 600 and directories 700.

7. COMMON SSH ERRORS

- Permission denied**
 - Authentication failed.
 - Wrong key or user.
 - Public key not in `authorized_keys`.
 - File permission issues.
- Connection refused**
 - SSH service not running.
 - Port is closed by firewall.
 - Wrong port.
 - Server is not listening on the expected address.
- Timeout**
 - Server is unreachable.
 - Network issue (routing, firewall).
 - Wrong IP address or hostname.
 - Security group blocking traffic.

8. EXAMPLE: VERBOSE SSH OUTPUT

Use `ssh -v` to see detailed connection steps.

```
[user@linux ~]$ ssh -v user@192.168.1.20
OpenSSH_9.6p1, debug1: Connecting to 192.168.1.20
debug1: Connection established.
debug1: Offering public key: /home/user/.ssh/id_rsa
debug1: Authentication succeeded (publickey).
debug1: Entering interactive session.
```

Look for where it fails. The verbose output shows exactly which step is failing.

9. SECURITY REMINDER



- Never expose your private SSH keys.
- Do not weaken file permissions as a shortcut.
- Use key-based authentication instead of passwords.
- Keep your SSH server updated.
- Use a non-default port only if necessary.
- Consider using a firewall and fail2ban.
- Remove unused or old keys.

Security shortcuts create real risks. Always follow best practices.

10. TROUBLESHOOTING FLOW

- 1 Check the SSH process (is it running?).
- 2 Check listening socket (`ss -lntp`).
- 3 Verify network connectivity (ping, traceroute).
- 4 Check authentication (keys, permissions).
- 5 Check logs (`journalctl -u ssh`).
- 6 Test again and verify shell access.

11. KEY TAKEAWAYS



- ✓ Understand the full SSH connection path.
- ✓ Use `ssh -v` for detailed debugging.
- ✓ Verify the SSH service is running and listening on the correct port.
- ✓ Check key files and permissions.
- ✓ Differentiate between permission denied, connection refused, and timeout.
- ✓ Always check logs for more information.
- ✓ Never expose your private keys or weaken permissions.
- ✓ A working SSH setup is essential for Linux administration and DevOps.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

PACKAGE AND DEPENDENCY PROBLEMS

UNDERSTAND • DIAGNOSE • FIX • AVOID INCIDENTS

1. WHAT PACKAGE MANAGERS DO

- Install, update, remove and manage software.
- Download packages from repositories.
- Resolve and install required dependencies.
- Keep track of installed versions.
- Help maintain a stable and secure system.



Package managers make software installation simple, but problems can occur when packages, repositories or dependencies are missing or incompatible.

2. KEY TERMS

Term	Meaning
Package	A compressed file that contains software and metadata (e.g. .deb, .rpm).
Repository	A trusted source that stores packages.
Dependency	Other packages required for a package to work.
Version	Specific release of a package (e.g. 1.2.3).
Package manager	Tool that installs and manages packages (e.g. apt, dnf).

3. DEBIAN/UBUNTU EXAMPLES (APT)

Update package list:

```
sudo apt update
```

Install a package:

```
sudo apt install <package-name>
```

Check installed version:

```
apt show <package-name>
```

Search for a package:

```
apt search <package-name>
```

Remove a package:

```
sudo apt remove <package-name>
```

Clean up unused packages:

```
sudo apt autoremove
```



4. RHEL-FAMILY EXAMPLES (DNF)

Update package list:

```
sudo dnf check-update
```

Install a package:

```
sudo dnf install <package-name>
```

Check installed version:

```
dnf info <package-name>
```

Search for a package:

```
dnf search <package-name>
```

Remove a package:

```
sudo dnf remove <package-name>
```

Show dependencies:

```
dnf repoquery --requires <package-name>
```



5. COMMON PROBLEMS AND ERRORS

Problem	Example Error Message
Package not found	E: Unable to locate package (package-name) No match for argument: (package-name)
Repository failure	Failed to fetch ... Could not resolve ... Repository is not reachable
Dependency conflict	Unmet dependencies. Dependency resolution failed.
Broken installation	dpkg was interrupted ... Error while installing ...
Version mismatch	Package (name) has no installation candidate Requires (package) >= X but Y is to be installed

6. TROUBLESHOOTING STEPS

- 1 Check the exact error message.
- 2 Verify the package name (spelling and version).
- 3 Make sure the repository is enabled and reachable.
- 4 Check for dependency issues.
- 5 Try clearing the cache and updating the package list.
- 6 Look for version conflicts.
- 7 If needed, use a specific version or enable the correct repository.
- 8 Read the logs and error output carefully.

7. USEFUL COMMANDS FOR TROUBLESHOOTING

Command	What It Does
<code>apt update / dnf check-update</code>	Refresh package lists.
<code>apt policy <package></code>	Show available versions (APT).
<code>dnf list <package></code>	Show available versions (DNF).
<code>apt --fix-broken install</code>	Fix broken dependencies (APT).
<code>dnf distro-sync</code>	Synchronize packages (DNF).
<code>apt clean / dnf clean all</code>	Clear package cache.
<code>journalctl -xe</code>	Check detailed system logs.

8. WHY BLINDLY UPGRADING CAN CREATE INCIDENTS

- New versions can introduce breaking changes.
- Dependencies may be incompatible with existing software.
- Critical services can stop working after an upgrade.
- Custom configurations may be overwritten.
- A working system can become unstable if not tested.

In production, always test upgrades in a non-production environment first.

9. BEST PRACTICES

- ✓ Use official and trusted repositories.
- ✓ Read release notes before upgrading.
- ✓ Test changes in a staging environment.
- ✓ Keep your system backed up.
- ✓ Install only required packages.
- ✓ Avoid unnecessary upgrades in production.
- ✓ Monitor services after installation or upgrade.
- ✓ Fix dependency issues properly, do not force install unless you understand the risk.

10. KEY TAKEAWAYS

- ✓ Understand how package managers work.
- ✓ Check repositories and dependencies.
- ✓ Read and understand error messages.
- ✓ Be careful with upgrades on production systems.
- ✓ Plan, test and monitor to avoid incidents.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

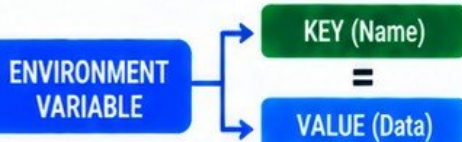
ENVIRONMENT VARIABLES, CONFIGURATION, AND APPLICATION FAILURES

UNDERSTAND • DIAGNOSE • FIX • APPLY EVERYWHERE

1. WHAT ARE ENVIRONMENT VARIABLES?

- Environment variables are key-value pairs that provide configuration and settings to applications, shells and operating systems.
- They can store paths, credentials, API keys, application settings and more.
- Applications read these variables at runtime to change their behavior.

Example: PATH tells the system where to find executable files.



Example: PATH=/usr/bin:/bin:/usr/local/bin

2. USEFUL COMMANDS

Command	What It Does
env	Shows all environment variables.
printenv	Shows all or a specific environment variable.
echo "\$PATH"	Displays the value of the PATH variable.
echo "\$VAR"	Displays the value of a specific variable.
set	Shows shell variables and environment variables.
unset VAR	Removes a variable from the current session.

```
[user@linux ~]$ echo "$PATH"
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin
```

3. CONFIGURATION FILES

- Applications often use configuration files to store settings.
- Common locations: /etc/, /etc/default/, /etc/sysconfig/, /opt/<app>/conf/, ~/.config/
- Configuration files can also contain environment variables.
- If values are missing, incorrect or have wrong paths, the application may fail.

Always check the application's documentation for the correct configuration file location.

Example configuration file: /etc/myapp/config.conf



```
DB_HOST=localhost
DB_USER=appuser
DB_PASSWORD=secret
LOG_LEVEL=info
```

4. COMMON ISSUES

Issue	Description
Missing variables	Required variable is not set.
Wrong paths	Path in variable or config file is incorrect.
Invalid configuration	Configuration file contains invalid values or syntax errors.
Incorrect credentials	Wrong username, password or API key.
Different shell environments	Variables set in one shell (e.g. .bashrc) may not be available in another (e.g. service).

5. SHELL ENVIRONMENTS

- Different shells and users can have different environment variables.
- Common files:
 - ~/.bashrc (user specific, non-login shell)
 - ~/.bash_profile (user specific, login shell)
 - /etc/profile (system wide, login shell)
 - /etc/environment (system wide variables)

A variable set in your terminal may not be available to services, cron jobs or other users.

6. SCENARIO: WORKS MANUALLY BUT FAILS AS A SERVICE

- You run the application manually and it works.
- When started as a service (e.g. systemd), it fails.
- Reason: the service does not have the same environment variables or configuration.
- Fix: define the required environment variables in the service configuration.

[Service]

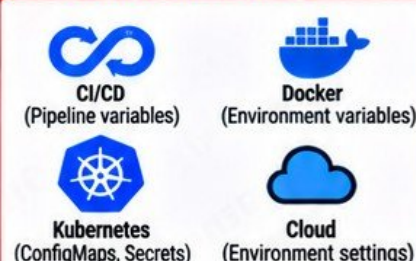
```
Environment=APP_ENV=production
Environment=DB_HOST=localhost
ExecStart=/opt/myapp/bin/start.sh
```

Use **systemctl daemon-reload** after changing the service file.

7. EXAMPLE TROUBLESHOOTING

- 1 Check if the variable is set: `echo "$VAR" or printenv VAR`
- 2 Check configuration file for correct values.
- 3 Verify file paths exist: `ls -l <path>`
- 4 Test credentials (database, API, etc.).
- 5 Compare environment between your shell and the service: `env (manual) vs systemctl show <service>`
- 6 Check service logs: `journalctl -u <service>`

8. DEVOPS CONNECTION



The same class of environment and configuration issues can cause failures in CI/CD pipelines, Docker containers, Kubernetes pods, and cloud workloads.

9. KEY TAKEAWAYS



- ✓ Understand what environment variables are and how they work.
- ✓ Use `env`, `printenv` and `echo` to check variables.
- ✓ Verify configuration files and paths.
- ✓ Check for missing or incorrect values.
- ✓ Remember different shells and services can have different environments.
- ✓ The same issues occur in CI/CD, Docker, Kubernetes and cloud environments.
- ✓ Always follow security best practices when handling credentials.

10. SECURITY REMINDER



- Do not hardcode sensitive values in scripts or configuration files.
- Use environment variables or secret managers.
- Never commit credentials to Git repositories.
- Restrict file permissions for sensitive files.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

LINUX TROUBLESHOOTING TOOLKIT



Same Questions.
Real Solutions.
Real Linux.

 **TROUBLESHOOT LIKE A PRO**
ASK THE RIGHT QUESTION.
USE THE RIGHT TOOL.

ONE QUESTION • THE RIGHT TOOL • FASTER ANSWERS • MORE CONFIDENCE

#	DIAGNOSTIC QUESTION	COMMAND(S)	EXAMPLE	WHAT IT SHOWS
1	Who am I?	whoami, id	\$ whoami \$ id	Shows the current user and user IDs (effective, groups).
2	Where am I?	pwd	\$ pwd	Shows the current working directory.
3	Does the file exist?	ls, stat, find	\$ ls /path/file \$ stat file \$ find / -name file	Checks if a file or directory exists and finds its location.
4	Can I access it?	ls -l, namei -l	\$ ls -l /path/file \$ namei -l /path/file	Shows permissions and checks access along the full path.
5	Is the process running?	ps, pgrep	\$ ps aux grep app \$ pgrep -fl app	Checks if a process is running and its PID.
6	Is the service healthy?	systemctl	\$ systemctl status nginx	Shows the status of a systemd service (active, inactive, failed, etc.).
7	What do the logs say?	journalctl	\$ journalctl -u nginx \$ journalctl -xe	Displays system and service logs for errors and clues.
8	Is CPU overloaded?	top, uptime	\$ top \$ uptime	Shows real-time CPU usage and system load average.
9	Is memory exhausted?	free -h	\$ free -h	Shows total, used, free memory, buffers/cache and swap.
10	Is storage full?	df, du	\$ df -h \$ du -sh /var/log	Shows disk usage and helps find large files or directories.
11	What is listening?	ss	\$ ss -lntp	Shows listening ports and which process is using them.
12	Does DNS work?	dig, getent	\$ dig example.com \$ getent hosts example.com	Tests DNS resolution and shows the resolved IP address.
13	Can the application respond?	curl	\$ curl -I http://localhost	Tests if the application is reachable and returns a response.

KEY LESSON



Choose a command because it answers a specific diagnostic question, not because it is the only command you know.

TROUBLESHOOTING MINDSET

- ✓ Start with a clear question.
- ✓ Use the right tool.
- ✓ Look at the output carefully.
- ✓ Narrow down the problem step by step.
- ✓ Combine tools when needed.
- ✓ Document what you find.
- ✓ Build your troubleshooting skills over time.

REMEMBER



- There is no single command for all problems.
- Different tools give different parts of the answer.
- Good troubleshooting comes from asking the right questions.
- Practice regularly to become faster and more confident.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Learn
Build
Grow

FINAL REAL-WORLD LINUX TROUBLESHOOTING LAB

INVESTIGATE • DIAGNOSE • FIX • VERIFY • THINK LIKE A DEVOPS ENGINEER

1. PRODUCTION ALERT



PRODUCTION ALERT

- Application unavailable
- HTTP 503
- Users cannot access the application

Your task is to investigate and fix the issue. You are not told the cause. Use a systematic troubleshooting approach.

2. ARCHITECTURE



3. INVESTIGATION COMMANDS (START HERE)

Run these commands on the Linux server and collect evidence.

1	uptime	# Check server load and uptime
2	df -h	# Check disk space
3	free -h	# Check memory usage
4	systemctl status nginx	# Check Nginx status
5	systemctl status myapp	# Check application service
6	journalctl -u myapp	# Check application logs
7	ss -lntp	# Check listening ports
8	curl localhost	# Test application locally

4. TROUBLESHOOTING FLOW

- 1 Observe**
Understand the problem and symptoms.
- 2 Collect evidence**
Run commands and check logs.
- 3 Isolate the failing layer**
Determine where the issue is (DNS, server, Nginx, app, database).
- 4 Form a hypothesis**
Based on the evidence.
- 5 Diagnose**
Find the real cause.
- 6 Fix**
Apply the correct fix.
- 7 Verify**
Confirm the application is working end-to-end.

5. EXAMPLE COMMAND OUTPUT (NORMAL)

```
$ uptime
10:24:17 up 2 days, 3:15,
load average: 0.12, 0.08, 0.05
```

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/xvda1     50G   12G   36G   25% /
```

```
$ free -h
total used free shared buff/cache available
7.6G   2.1G   4.8G   512M   688M   5.1G
```

```
$ systemctl status nginx
● nginx.service - A high performance web server
   Active: active (running) since Tue 2024-08-27 10:00:00
     Main PID: 1234 (nginx)
```

```
$ systemctl status myapp
● myapp.service - My Application
   Active: failed (Result: exit-code) since Tue 2024-08-27 10:05:00
     Main PID: 5678 (code=exited, status=1)
```

```
$ journalctl -u myapp
Aug 27 10:05:01 server myapp[5678]:
Error: Cannot connect to database
Aug 27 10:05:01 server myapp[5678]:
FATAL: database connection failed
```

```
$ ss -lntp
State  Recv-Q  Send-Q  Local Address:Port
LISTEN  0        128     0.0.0.0:80
users: ("nginx",pid=1234)
LISTEN  0        128     127.0.0.1:3000
users: ("myapp",pid=5678)
```

```
$ curl localhost
HTTP/1.1 503 Service Unavailable
Server: nginx/1.24.0
Content-Type: text/html
<html><body><h1>503 Service Unavailable
</h1></body></html>
```

7. POSSIBLE ROOT CAUSES

- Application service is down or crashing.
- Database connection failure.
- Incorrect configuration (wrong DB host, port, credentials).
- Nginx misconfiguration.
- Port not listening.
- Disk full (logs or application data).
- Memory exhausted (OOM).
- Dependency or package issue.
- Recent change or deployment introduced a bug.

8. APPLY THE FIX

- ✓ Fix the configuration or credentials.
- ✓ Start or restart the service.
- ✓ Free disk space if needed.
- ✓ Increase memory or fix memory leak.
- ✓ Correct Nginx configuration if required.
- ✓ Verify database connectivity.
- ✓ Re-deploy a working version if necessary.

9. VERIFY THE SOLUTION

- ✓ curl localhost returns 200 OK.
- ✓ Application is accessible through Nginx.
- ✓ No more 503 errors.
- ✓ Check logs for new errors.
- ✓ Monitor for a few minutes.
- ✓ Test from an external machine (if possible).

10. KEY TAKEAWAYS

- ✓ Always start with evidence.
- ✓ Follow a systematic approach.
- ✓ Each command answers a specific question.
- ✓ Isolate the failing layer.
- ✓ Do not assume, verify.
- ✓ The same troubleshooting process applies in CI/CD, Docker, Kubernetes and cloud environments.

"Good engineers guess. Great engineers verify."

Same mindset. Bigger systems.
Linux skills scale everywhere.

Troubleshooting is a skill.
Practice makes you a better engineer.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Learn
Build
Grow